HAL open science

Safety analysis integration in a SysML-based complex system design process

F. Mhenni, H. Kadima, J. Choley

To cite this version:

F. Mhenni, H. Kadima, J. Choley. Safety analysis integration in a SysML-based complex system design process. 2013 7th Annual IEEE Systems Conference (SysCon), Apr 2013, Orlando, France. pp.70-75, 10.1109/SysCon.2013.6549861 . hal-01887992

HAL Id: hal-01887992

https://hal.science/hal-01887992v1

Submitted on 7 Jun 2023

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Autorisation HAL

HAL Authorization

Safety Analysis Integration in a SysML-Based Complex System Design Process

Faïda Mhenni*, Nga Nguyen†, Hubert Kadima† and Jean-Yves Choley*
*SUPMECA Paris, Saint-Ouen, France
Email: faida.mhenni, jean-yves.choley@supmeca.fr
†EISTI, Cergy, France
Email: nn, hk@eisti.eu

***Abstract*—Model-based system engineering is an efficient approach to specifying, designing, simulating and validating complex systems. This approach allows errors to be detected as soon as possible in the design process, and thus reduces the overall cost of the product. Uniformity in a system engineering project, which is by definition multidisciplinary, is achieved by expressing the models in a common modeling language such as SysML. This paper presents an approach to integrate safety analysis in SysML at early stages in the design process of safety-critical systems. Qualitative analysis is performed through functional as well as behavioral safety analysis and strengthened by formal verification method. This approach is applied to a real-life avionic system and contributes to the integration of formal models in the overall safety and systems engineering design process of complex systems.**

I. Introduction

Over the last decade, the complexity of industrial systems has considerably grown since they integrate an increasing number of components and a variety of technologies. Meanwhile, system engineers always have to reach the following main objectives: building the right systems, building them correctly and on time, while reducing costs. Thus, model-based systems engineering approach is a necessity in system design to better manage these constraints. SysML [1], [19] is a semi-formal unified language dedicated to modeling systems. It allows engineers to document the properties from different disciplines and to describe the whole solution [13]. This OMG standard is becoming more and more supported by industry because it provides a consistent, well-defined, and well-understood language to communicate the requirements and corresponding designs among engineers.

In addition to being complex, industrial systems are also safety-critical. Hazard and risk analyses are critical to guarantee the reliability, robustness, and quality of products. In general, safety analysis techniques can be split into two categories: qualitative and quantitative approaches. Qualitative methods try to find the causal dependencies between a hazard on system level and failures of individual components, while quantitative methods aim at providing estimations about probabilities, rates and severity of consequences. These safety analyses are usually performed separately with independent tools. Consequently, they occur late in the design process when the design is already finalized and thus, miss the opportunity to influence design choices and decisions [23]. The purpose of this paper is to provide a methodology based on pertinent semi-formal and formal models to automate parts of the safety analysis process and, consequently, both reduce the cost and improve the quality of the system safety studies. The methodology allows system engineers to perform early validation of system safety requirements in the design process by using model checking [6], a formal verification method. Safety requirements are integrated in SysML requirement diagrams that are then verified by formal test cases whose scenarios can be described by SysML state-machine diagrams, extended with failure modeling. Given the formal model of the system, the safety analysis process consists of defining a set of formal properties via temporal logic formulas to represent the informal safety requirements and then using a model checker to determine whether the proposed system architecture satisfies the safety requirements. The tight coupling between the two environments, the semi-formal and the formal one, to provide a unified framework for model-based safety analysis and to ensure the traceability between system design and safety models of complex systems is the main contribution of our work.

The rest of the paper is organized as follows. Section II talks about related work in the domain of safety analysis and system engineering. Section III presents our methodology with different steps to carry out a unified model-based safety analysis in the whole system engineering process. Section IV describes in detail our case study, which is the aircraft wheel brake system, with different SysML diagrams as well as the application of the NuSMV model checker to verify some important properties extracted from safety requirements. Conclusion is given in Section V.

II. Related Work

During the safety assessment process of a system, a number of interrelated analysis methodologies are used. Most commonly used hazard analysis methodologies in the discipline of system safety (e.g. Preliminary Hazard Analysis, System Hazard Analysis, Fault Tree Analysis, Event Tree Analysis, Failure Mode and Effects Analysis, Functional Hazard Analysis, Petri Net Analysis or Markov Analysis) are well described in a book written by Ericson [10]. Each hazard analysis technique is a unique analysis methodology using specific guidelines and rules with an overall objective of identifying

hazards, mitigating them, and assessing system residual risk. To perform safety analyses, the two most traditionally used fault modeling techniques are Failure Mode and Effects Analysis (FMEA) and Fault Tree Analysis (FTA). FMEA aims to evaluate the effects of potential failure modes of components or functions, and eliminate these potential risks in the system design. Meanwhile, FTA is a top-down, deductive analytical method in which initiating primary events such as component failures, human errors, and external events are traced through Boolean logic gates to an undesired global system-level event. Both of these techniques are compositional analyses based on the topology of the system and do not consider its dynamic behavior. For a comprehensive safety analysis however, the behavioral aspect should also be considered.

Model checking is a formal verification method used to verify a set of desired behavioral properties (usually related to safety requirements) of a system through exhaustive enumeration of all the states reachable by the system and the behavior that goes through them. This automated process receives a model of the system and one or more temporal logic formulas [14] representing the properties to be verified, and determines whether the system satisfies these properties or not, in which case a counter example (a path leading to undesired state) is generated to help the designer perform corrective design changes. The application of model checking in safety analysis has been studied in many researches [4],[5], [15], [22], [23] for safety requirements verification and/or automatic fault trees generation. Two different well known tools can be used for model checking: FSAP/NuSMV-SA and AltaRica.

FSAP/NuSMV-SA [5] is an automated analysis tool that consists of two components: FSAP (Formal Safety Analysis Platform), which provides a graphical user interface, and NuSMV-SA, which is based on the NuSMV model checker as safety analysis engine. The tool provides some predefined failure modes that can be injected by safety engineers in the initial model of the system to create a so-called extended system model. The safety requirements are expressed in temporal logic and can be subsequently verified using the NuSMV model checking verification engine. The model checker is used as a validation tool, or a powerful fault tree analysis tool. However, one limitation of this tool is that the fault trees automatically generated with minimal cut sets have a flat structure with only two levels deep. This representation does not reflect the structure of the system and consequently, the fault tree comprehension is not very intuitive for system engineers.

AltaRica [21], [2], [4] is a formal specification language that was designed to specify the behavior of complex systems. An AltaRica model is composed of nodes that are characterized by their reachable states, in and out flows, events, transitions and assertions. Once a system model is specified in the AltaRica language, it can be compiled into a lower level formalism such as finite-state machines, fault trees and stochastic Petri Nets. The safety requirements are formalized with the use of linear temporal logic operators and the formal verification technique can be performed by the AltaRica's MEC model checker. Nevertheless, this model checker is limited by the size of systems it can handle.

The recent work of [23] dealt with the integration of the Compositional Safety Analysis (CSA) and the Behavioral Safety Analysis (BSA). First, system failure models such as Fault Tree Analysis and Failure Mode and Effects Analysis are constructed by establishing how the local effects of component failures combine as they propagate through the hierarchical structure of the system. The technique used for CSA is called HiP-HOPS (Hierarchically Performed Hazards Origin and Propagation Studies), proposed in [20]. CSA gave preliminary information about state-automata that represent the transition between normal and failure states of the system. Next, in the BSA, model checking can be carried out on these behavioral models in order to verify automatically the satisfaction of safety properties. So the CSA and BSA could be effectively combined to benefit from the advantages of both approaches. Even so, behavioral information captured from CSA is rather limited because its main purpose is the failure propagation and hierarchy, not the dynamic behavior.

Joshi et al described in their report [15] the so called ModelBased Safety Analysis in which the nominal (non-failure) system behavior captured in model-based development is augmented with the fault behavior of the system. Like in [23], temporal logic is used to formalize informal safety requirements, and the model checker NuSMV is used to validate these requirements. To illustrate the process, a case study about the wheel brake system is well detailed in the report, with a fault model consisting of different component failures, i.e. digital and mechanical failure modes. Fault tolerance verification is carried out by using additional variables and real-time temporal logic operators to investigate if the system can handle some fixed number of faults. Nonetheless, the model-based development studied in this paper addresses principally Simulink. In our paper however, we are interested in integrating safety analysis into a more general framework, namely system engineering via SysML.

Laleau et al. [16] tried to combine SysML requirement diagrams and the B formal specification language. Since requirements in SysML are textual, the SysML requirement models are firstly extended to represent some concepts in the goal-oriented requirement engineering approach, such as expectation, elementary or abstract goal for requirement classes and milestone, and/or refinement for relationship between requirements. Then, derivation rules are proposed to translate the SysML goal models into B specifications. By doing this, a more precise semantics of SysML goal models is given, narrowing the gap between the requirement phase and the formal specification.

Also regarding requirements, Dubois [9] proposed to directly include system requirements in the design process but the separation with the proposed solutions as required by safety standards is achieved by isolating the following triplet: requirement models, solution models and validation and verification models. A SysML profile respecting safety standards called RPM (Requirement Profile for MeMVaTEX)

has been developed. The requirement stereotype of SysML is replaced by the MeMVaTEX requirement, by adding various properties such as verifiable, verification type, derived from, satisfied by, refined by, traced to, etc. So, the traceability is assured between requirement models, between requirement and solution models, and between requirement and V&V models by using these properties.

Another approach to integrate SysML and safety analysis is the use of the common modeling framework Eclipse [25]. In this work, an independent tool called Obeo Designer Safety viewpoint that implements classical risk analyses is developed. Then the interoperability of this modeling tool and the SysML model is achieved through the Eclipse Modeling Framework. To make the integration possible, the authors used the open source SysML Topcased editor. Safety elements can reference SysML model elements since they are both expressed in the same framework. Furthermore, the Topcased GenDoc plugin can also be used to generate safety documentation from the two models. So in this approach, SysML is not extended with a safety profile in order to tune the SysML models. In a more recent work [3], a translation from Obeo Designer's Domain Specific Language for FMEA (Failure Mode and Effects Analysis) and PHA (Preliminary Hazard Analysis) into AltaRica is added to enable formal verification. However, no real system has been studied yet in order to prove the scalability of the method.

David et al. [7], [8] worked on the generation of an FMEA report from system functional behaviors written in SysML models, and on the construction of dysfunctional models by using the AltaRica language in order to compute reliability indicators. In their methodology called MéDISIS, they start with the automatic computation of a preliminary FMEA. The structural diagrams, namely Block Definition Diagram (BDD) and Internal Block Diagram (IBD), and the behavioral diagrams such as Sequence Diagram (SD) and Activity Diagram (AD) are analyzed in detail to give an exhaustive list of failure modes for each component and each function, with their possible causes and effects. Then the final FMEA report is created with help from experts in the safety domain. To facilitate a deductive and iterative method like MéDISIS, a database of dysfunctional behaviors is kept updated in order to rapidly identify failure modes in different analysis phases. The next step of their work is the mapping between SysML models and AltaRica data flow language, so that existing tools to quantify reliability indicators such as the global failure rate, the mean time to failure, etc can be used directly on the failure modes identified in the previous step.

Garro et al. [11] developed RAMSAS, a model-based method for system reliability analysis that combines SysML and the Simulink tool allowing the verification of reliability performance of the system though simulation. A formal verification method was not used in this research for safety assessment.

There are a lot of works about model-based safety analysis but few researches have addressed the direct connection between SysML and model checking. This paper focuses on a methodology that integrates formal safety analysis directly in SysML in order to provide a unified framework with better traceability between safety and system models. Our methodology will be described in the next section.

III. Integrating safety analysis in the design process

Traditionally, safety analysis is only performed at a late stage of the design process, and thus does not directly influence system design. The aim of our work is "design safe" by integrating safety measure as soon as possible while designing a new system.

The design process usually starts with capturing initial requirements and then modeling system functions. At this early stage, when a functional model is established, functional safety analysis (using Functional Hazard Analysis (FHA) or Functional Failure Mode and Effects Analysis (FMEA) for example) can already be performed to assess the failure effects of each function. According to the severity of failure effects, system functions are classified as critical or not. This classification allows appropriate resource and priority allocation to each function. Focus is placed on critical functions whose failure may lead to catastrophic effects. Necessary design modifications shall be performed at this stage to suppress or mitigate identifiend potential risks. New functions such as fault detection may be introduced and the process shall be iterated until a satisfactory functional model is established. This analysis is performed based on the SysML functional model of the system and consequently ther is no gap between the system model and safety analysis.

Then, components are allocated to functions to define the system structure. A Block Definition Diagram is used to model the system structure in SysML. Alternative solutions are identified and are compared before a final solution is chosen. Integrating safety analysis at this level allows system designers to consider safety among the trade-off criteria. Component FMEA is then realized to each component of the system (components are picked up from the corresponding Block Definition Diagram). Failure propagation can be described in a fault tree established based on the Internal Block Diagram, which illustrates the interactions between the different components in the SysML model. Based on these studies, the design may be modified to take into account safety aspects from early design stages, minimizing late and costly design changes. A safer design can be obtained by adding backup and redundant components. System behavior is then refined by the choice of redundancy approaches (static, dynamic or hybrid) and how the system shall behave in case of failure. New safety requirements are derived from the analyses performed above and the SysML requirement model is updated to integrate these new requirements. Like for the functional safety analysis, the component based safety analysis is performed based on the system's SysML model and thus is directly linked with the latest version of the model. Design changes on components are also directly integrated in the SysML model improving the whole consistency. More detail with a case study example

about preliminary safety analysis based on functional and structure models of the system can be found in our previous work [17].

At this stage, a structural and behavioral description of the system is already established and safety requirements have been derived and/or refined from other requirements (stakeholder or higher level). Measures have been taken to mitigate the potential risks identified in compositional safety analyses above, by adding new and/or modifying exixting functions/components. The next step consists in verifying that, during its operational phase, the system respects safety requirements. In other words, the dynamic behavior of the system shall also be satisfy a set of safety requirements mainly requirements about the number of tolerated failures and how the system shifts to degraded states before failure. Test cases can be defined in SysML and linked (using the *verify* relationship) to the corresponding requirements. In our study, formal test cases carried out by a model checking tool have been chosen to assess requirements satisfaction. The advantage of formal verification tests is that they execute all possible computation paths corresponding to different combinations of states the system can have. This task cannot be done manually, especially for a large number of states. We shall make sure that all requirements are linked to test cases and thus will be verified. SysML provides automatically generated traceability diagrams and tables to list all the requirements and their links to other model elements.

Formal verification is based on the following elements that have already been established:

- System requirements (including safety requirements) that are modeled in SysML language via Requirements and Requirement Diagrams to illustrate relationships between requirements;
- System functions and the flow exchanges among them given by system Use Cases and Activity Diagrams in SysML;
- A structural description of the system given by Block Definition Diagram and Internal Block Diagram in SysML. The first diagram describes the composition of the system while the latter illustrates interactions and flow exchanges between components inside the system.

Traceability links are established between the model elements (requirements, functions, blocks and test cases) in SysML to facilitate verification tasks.

Given these elements, an extended formal model taking into account failures and their impact on the system is built. A SysML State Machine Diagram can be used for this purpose since it can model the dynamic behavior of the system in presence of faults. Different system states (or modes) and the transitions (including failure occurrences) are thus modeled. This model is then mapped to a formal model abstracting the considered system, written in an appropriate language accepted by a model checker. NuSMV has been selected in our study because this symbolic model checker is more scalable and therefore is recommended for large real-life systems. Moreover, its availability as an open-source program makes it easier for future integrated framework tool development. The NuSMV model consists in a main module that describes the whole system states, and a specific module for each component (or part in the SysML Model). The informal safety requirements are also translated into formal properties using temporal logic formulas. Running the corresponding program assesses whether the system satisfies these properties or not. If one or more properties are not validated, the model checker gives a counter example which will help to find out the causes of the problem. In this case, system engineers have to revise the system design until a new solution fulfilling all safety requirements is found.

IV. Case study

A. Introduction

The case study addressed in our paper is the wheel brake system of an aircraft described in the ARP 4761 standard Appendix L [24], and also studied in [12], [15] and [23]. This wheel brake system together with other aircraft sub-systems contributes to achieving the system function: ”Decelerate Aircraft on Ground”. This function is proved to be safety critical. Elements of safety analysis will be given in section IV-B.

B. Safety Analysis

A top-down design process usually begins with requirements definition and analysis. Then a breakdown of system functions is established. A functional decomposition of the aircraft functions is given in the ARP 4761 document [24]. In this work, we will focus on the sub-function ”Decelerate Aircraft on Ground” of the high level function ”Control Aircraft on Ground”.

At this early stage, a functional safety analysis can be performed by analyzing the failure effects of each function. Safety experts shall consider all possible scenarios of failure and the effects of each potential failure on the system and its user(s). To help capitalization and reuse, different level function libraries can be integrated in SysML, with their respective failure modes and effects. This library shall be updated when new elements are availabe. Based on this FMEA, a classification of the functions is proposed according to the severity of failures modes.

A preliminary safety analysis of the ”Decelerate Aircraft on Ground” shows that the considered function is critical to the functioning of the system since its failure could lead to catastrophic consequences like the aircraft leaving the runway or crashing the buildings or equipment on the airport. A safety requirement is derived from this analysis: ”Loss of wheel braking during landing or rejected take-off shall have a frequency less than 5 E-7 per flight” [15] (SR).

A further breakdown of the ”Decelerate Aircraft on Ground” shows that it has four sub-functions that are: ”Decelerate Wheels on Ground”, ”Prevent Aircraft from Moving when Parked”, ”Control Thrust Reverser” and ”Control Ground Spoiler”. Since the wheels deceleration has a stronger effect than other braking systems, our safety analysis will be hold

on the function ”Decelerate Wheels on Ground”. The wheel braking system, which is allocated from this function, must be fault tolerant. This property is obtained by using redundant components in our example. The braking system is then made of a physical part that actually brakes the wheels and a control part to monitor the functioning. Redundancy is applied to both parts. The physical part contains two redundant hydraulic lines, a Normal line that is first activated and an Alternate one that is activated when the normal chain is inoperative. Each of the two systems has an independent power source. A supplementary power source, called emergency power source, is also mandatory for the wheel-brake system in aircraft [18]. In our case, an accumulator is also added and provides the braking system with hydraulic power when all the other power sources are inoperative. The SysML Internal Block Diagram in Fig.1 describes the wheel brake system and the interactions between components. The interactions help analyzing the fault propagation through the system.

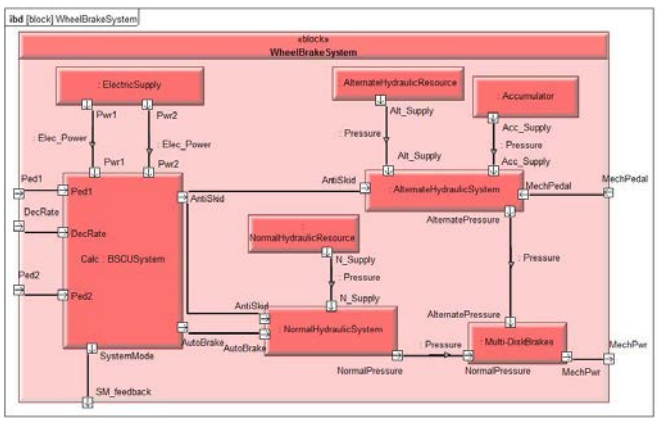

Fig. 1. Wheel Brake System Structure

To model this behavior, we consider four modes or states of the system that are:

- Normal: when the Normal line is operated;
- Alternate: activated when the Normal line fails and this failure is noted as **failNormal**;
- Emergency: activated when Alternate mode also fails (**failAlternate**). The accumulator provides power source in this mode;
- Fail: when all systems are in failure. The failure of the accumulator is noted as **failEmergency**.

These modes and the corresponding transitions are modeled in an automata describing the dynamic behavior of the system including fault effects given by a SysML State Machine Diagram (see Fig.2).

To satisfy the safety requirement SR, the wheel brake system shall be fault tolerant, i.e. it shall be able to brake the system even in presence of a certain number of failures. This can be translated, for instance, into the two safety requirements. These requirements are linked to corresponding test cases in a SysML Requirements Diagram (Fig.3).

- SR1-WBS: When output is not supplied by Normal Line and there is no failure accounted in Alternate line, pressure shall be supplied from Alternate line;
- SR2-WBS: When both Normal line and Alternate line are not producing output, as long as there is no failure accounted along the emergency line, the system shall not fail.

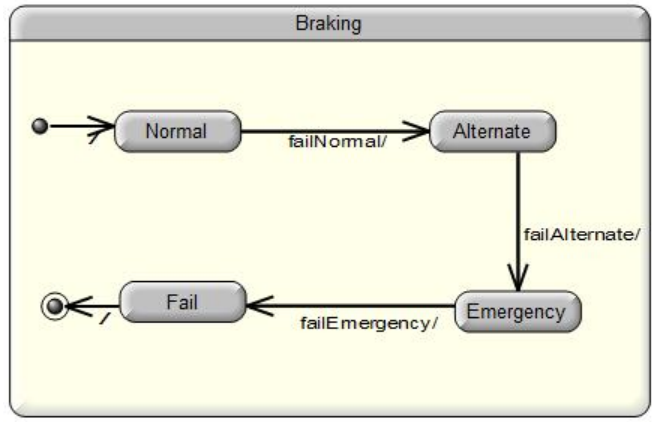

Fig. 2. Dynamic Behavior Including Faults Automata

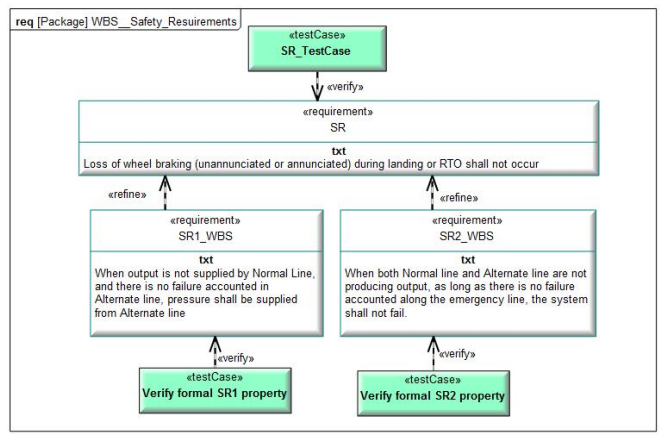

Fig. 3. Safety Requirements in SysML

Formal verification is chosen to verify these requirements. For this the automata describing the dynamic behavior of the system in Fig.2 together with the requirements to be verified shall be translated into a NuSMV program. The safety requirements SR1-WBS and SR2-WBS are respectively specified by temporal formulas F1-WBS and F2-WBS:

- F1-WBS: SPEC AG ((*Normal.output* = False AND *Alternate.failAlternate*= False)

  $\rightarrow$

  Alternate.output=True).
- F2-WBS: SPEC AG ((*Normal.output* = False) AND (*Alternate.output* = False) AND (*Accumulator.failEmergency* = False)

  $\rightarrow$

!(*systemMode* = Fail))

Running the NuSMV program, these two properties are shown to hold.

V. Conclusion

In this paper, we introduced an integrated methodology that combines SysML safety analysis with a formal verification method, namely model checking. Model-based system engineering analysis via SysML aims to facilitate the communication among interdisciplinary engineers by employing a consistent and well-defined language. Joining the safety analysis together with the design phase through SysML models will enhance consistency in the whole system, enable earlier error detection, avoid expensive re-design and reduce time to market delays. Another advantage of SysML is that it is not a specific tool or methodology. The combination of safety analysis with a standard modeling language instead of some specific tools or languages allows us to achieve wider analysis coverage and a more robust assessment. Furthermore, the traceability between system and safety models, i.e. the safety requirements and the formal test cases, assures a coherent safety analysis, without semantic rupture in the whole process of system engineering.

In our previous work [17], we have shown how some safety analysis results such as FMEA can be generated from SysML structural and behavioral diagrams. In this paper, we extend the work by integrating in this unique framework the automated verification of formal models with regard to safety properties. However, The translation of informal safety requirements into temporal logic formulas is not always straightforward.

Our future work is to investigate more in detail the structure of the system to propose a more precise behavioral formal model with different failure modes propagation. Input and output flows as well as internal failure modes will be taken into account. Complex system properties for verifying system fault tolerance by using special temporal logic operators must be supported. At the same time, the scalability of the model checker must be tested in accordance to the size and the complexity of the models.

References

[1] www.omgsysml.org.

[2] A. Arnold, A. Griffault, G. Point, and A. Rauzy, "The AltaRica language and its semantics," *Fundamenta Informaticae*, vol. 34, pp. 109–124, 2000.

[3] F. Belmonte and E. Soubiran, "A model based approach for safety analysis," in *Computer Safety, Reliability, and Security*, ser. Lecture Notes in Computer Science, F. Ortmeier and P. Daniel, Eds. Springer Berlin Heidelberg, 2012, vol. 7613, pp. 50–63.

[4] P. Bieber, C. Castel, and C. Seguin, "Combination of fault tree analysis and model checking for safety assessment of complex system," in *Proceedings Fourth European Dependable Computing Conference (EDCC4). Toulouse (France).*, October 2002, pp. 19–31.

[5] M. Bozzano and A. Villafiorita, "Improving system reliability via model checking: The FSAP/NuSMV-SA safety analysis platform," in *Computer Safety, Reliability, and Security*, ser. Lecture Notes in Computer Science, S. Anderson, M. Felici, and B. Littlewood, Eds. Springer Berlin Heidelberg, 2003, vol. 2788, pp. 49–62.

[6] E. M. Clarke, J. O. Grumberg, and D. A. Peled, *Model Checking*. Cambridge: The MIT Press, 1999.

[7] R. Cressent, V. Idasiak, and F. Kratz, "Prise en compte des analyses de la sûreté de fonctionnement dans l ingénierie de système dirigée par les modèles SysML," *Gnie Logiciel*, pp. 33–39, 2011.

[8] P. David, "Contribution à l'analyse de sûreté de fonctionnement des systèmes complexes en phase de conception : application à l'évaluation des missions d'un réseau de capteurs de présence humaine," Ph.D. dissertation, Université d'Orléans, Novembre 2009.

[9] H. Dubois, "Gestion des exigences de sûreté de fonctionnement dans une approache IDM," *Génie logiciel*, no. 85, pp. 14–19, Juin 2008.

[10] C. A. Ericson, *Hazard Analysis Techniques for System Safety*. John Wiley & sons, 2005.

[11] A. Garro and A. Tundis, "Enhancing the RAMSAS method for system reliability analysis - an exploitation in the automotive domain," in *SIMULTECH*, 2012, pp. 328–333.

[12] R. Guillerm, "Intégration de la sûreté de fonctionnement dans les processus de l'ingénierie système," Ph.D. dissertation, Université de Toulouse, 2011.

[13] M. Hause and F. Thom, "Building bridges between systems and software with SysML and UML," in *INCOSE Intl. Symposium, June*, 2008.

[14] M. Huth and M. Ryan, *Logic in Computer Science : Modelling and Reasonning about Systems*. Cambridge University Press, 2004.

[15] A. Joshi, M. P. E. Heimdahl, S. P. Miller, and M. W. Whalen, "Model-based safety analysis," Nasa Langley Research Center, Contractor Report, February 2006.

[16] R. Laleau, F. Semmak, A. Matoussi, D. Petit, A. Hammad, and B. Tatibouet, "A first attempt to combine SysML requirements diagrams and B," *Innovations in Systems and Software Engineering*, vol. 6, pp. 47–54, 2010.

[17] F. Mhenni, J.-Y. Choley, A. Rivière, N. Nguyen, and H. Kadima, "SysML and safety analysis for mechatronic systems," in *MECATRONICS REM*, 2012.

[18] I. Moir and A. Seabridge, *Aircraft Systems, Mechanical Electrical and Avionics Subsystems Integration*, 2nd ed. Professionnal Engineering Publishing, 2001.

[19] *OMG Systems Modeling Language (OMG SysML)*, OMG Std., June 2012.

[20] Y. Papadopoulos and J. A. McDermid, "Hierarchically performed hazard origin and propagation studies," in *SAFECOMP 99, 18th International Conference on Computer Safety, Reliability and Security, Toulouse France*, ser. Lecture Notes in Computer Science, vol. 1698. Springer Verlag, 1999, pp. 139–152.

[21] G. Point, "AltaRica : Contribution à l unification des méthodes formelles et de la sûreté de fonctionnement," Ph.D. dissertation, LUniversit Bordeaux I, 2000.

[22] C. Seidner, "Vérification des EFFBDs : Model checking en ingnierie système," Ph.D. dissertation, Université de Nantes, 2009.

[23] S. Sharvia, "Integrated application of compositional and behavioural safety analysis," Ph.D. dissertation, The University of Hull, February 2011.

[24] *Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment*, Society of Automotive Engineers Std., 1996.

[25] F. Thomas and F. Belmonte, "Performing safety analyses and sysml designs conjointly : a viewpoint matter," in *Complex Systems Design & Management*, 2011.